Prérequis

Installez votre travail précédent (ou la correction) dans votre environnement de travail.
N'oubliez pas d'importer la base de données fournie.

Entrées, validation et échappement à l'affichage

Le fait d'utiliser la méthode `prepare` de PDO nous prévient déjà des *injections SQL* (les valeurs liées ne sont pas interprétées comme du code SQL). En revanche, cela ne protège **pas** contre les failles *XSS* (*Cross-Site Scripting*) : si une donnée provenant de l'utilisateur (ou lue en base) est injectée telle quelle dans une page HTML, elle peut être exécutée par le navigateur.

Rappelez-vous que toutes les données reçues via `$_GET` ou `$_POST` sont des chaînes de caractères et peuvent contenir n'importe quoi — y compris du HTML ou du JavaScript malveillant.

Échapper à l'affichage avec `htmlspecialchars`

Pour afficher une donnée dans une page HTML en toute sécurité, on transforme les caractères spéciaux HTML (`<`, `>`, `"`, `'`, `&`, …) en entités affichables. La fonction PHP adaptée est :

- `htmlspecialchars($aString, $option, $encoding)` — l'option `ENT_QUOTES` convertit aussi les quotes simples et doubles ; on utilise l'encodage `"UTF-8"`.

```php
<?php
    $str = "<p>Bonjour</p>";
    echo $str;                   // le navigateur interprète un vrai
paragraphe
    echo htmlspecialchars($str, ENT_QUOTES, "UTF-8");
                                 // affiche le texte littéral <p>Bonjour</p>
                                 // car le HTML reçu par le navigateur est
                                 // "&lt;p&gt;Bonjour&lt;/p&gt;"
?>
```

> **Note**
>
> Dans certains contextes, on voudrait *supprimer* les balises HTML plutôt que de les échapper. La fonction `strip_tags($aString)` enlève les balises, mais **elle ne suffit pas** à prévenir les XSS (elle ne traite pas correctement les quotes, etc.) et **elle altère** la donnée. Ce n'est donc **pas** la bonne approche pour stocker une donnée en base, ni un substitut à l'échappement à l'affichage.
>
> Exemple du problème avec `strip_tags` seul :

```php
<?php
    $str = "<b>myvalue</b>\" oninput=alert(1) \"";
    $str = strip_tags($str);

    echo "<input type=\"text\" value=\"$str\">";

    // produit <input type="text" value="myvalue" oninput=alert(1) "">
    // qui lance un popup à chaque frappe.
?>
```

Ne pas altérer les données en entrée

Mauvaise pratique (à éviter) : appeler `htmlspecialchars` ou `strip_tags` **avant** d'enregistrer une valeur en base. On perd de l'information et on mélange « stockage » et « présentation ».

Dans ce cours, on adopte les règles suivantes :

1. **Entrée** : on se limite à un `trim(...)` (espaces accidentels en trop) et, le cas échéant, à une **validation** (format, taille minimale, mots de passe identiques, etc. — voir l'inscription en 03c). La base de données stocke la donnée **brute**.
2. **Affichage** : on échappe **uniquement** au moment d'injecter la donnée dans le HTML.

La fonction `escape`

Ajoutez dans `functions.php` la fonction suivante. Le transtypage `(string)` évite les avertissements de dépréciation de PHP 8.2+ lorsque la valeur est `null` :

```php
function escape($str) : string {
    return htmlspecialchars((string)$str, ENT_QUOTES, "UTF-8");
}
```

Utilisez `escape` **partout où vous affichez** une donnée dynamique dans le HTML (texte, attributs `value`, `src`, `alt`, titres, etc.).

Exemple pour le pseudo reçu en GET dans `profile.php` :

- **Entrée** (après vérification que le paramètre existe) :

```php
$pseudo = trim($_GET["pseudo"]);
```

- **Affichage** (title, bandeau, description, image, lien Home, …) : enveloppez avec `escape(...)`, par exemple `<?= escape($pseudo) ?>` ou `echo escape($description);`.

Appliquez dès à présent `trim` en entrée et `escape` à l'affichage dans `profile.php` (et dans la suite du parcours pour toutes les pages concernées).

Page de login

Attention

Cet exercice est beaucoup moins dirigé que les précédents, à vous d'écrire le code nécessaire pour le réaliser.

Ouvrez le fichier `login.php`. Remarquez qu'il s'agit d'un formulaire qui demande à un utilisateur son pseudo et son mot de passe. Ce formulaire envoie ces informations à l'aide de la méthode *POST* à la même ressource, c'est à dire `login.php`.

Sur base de ce que vous avez appris jusqu'ici, modifiez ce fichier afin d'afficher un message d'erreur et de reproposer une identification si le nom d'utilisateur et/ou le mot de passe est incorrect. Si le couple utilisateur et mot de passe est correct, redirigez l'utilisateur vers la page de son profil.

Rappels :

- en **entrée**, appliquez `trim` aux valeurs de `$_POST` (pseudo et mot de passe) — **pas** d'échappement HTML avant la comparaison avec la base ;
- à l'**affichage**, utilisez `escape` pour les attributs `value` du formulaire (réaffichage après erreur) et pour toute partie dynamique d'un message (par ex. le `$pseudo` dans le texte d'erreur).

Pour gérer les erreurs, créez une variable `$error` si vous rencontrez une erreur et remplissez-la avec un message explicatif. Par exemple :

- `"Can't find a member with the pseudo '$pseudo'. Please sign up."`
- `"Wrong password. Please try again."`

Cette variable `$error`, si elle existe, devra être affichée juste après le formulaire, dans un `<div>` de classe `errors` (en échappant correctement le contenu dynamique) :

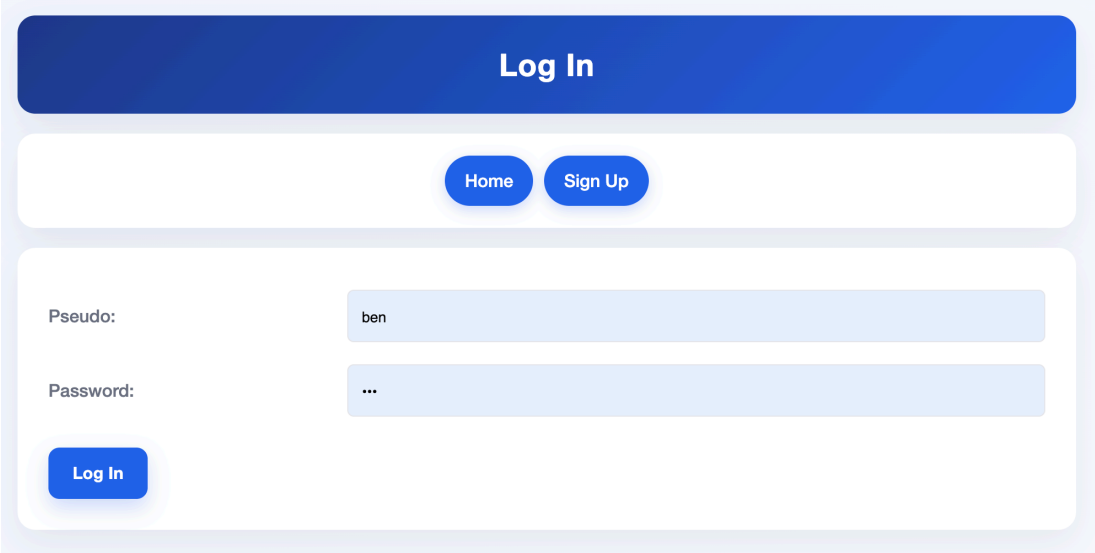
```php
<?php
    if (isset($error))
        echo "<div class=\"errors\">$error</div>";
?>
```

Pour vous aider, voici une fonction qui permet de rediriger le client vers une autre page. Ajoutez-la à votre fichier `functions.php`.

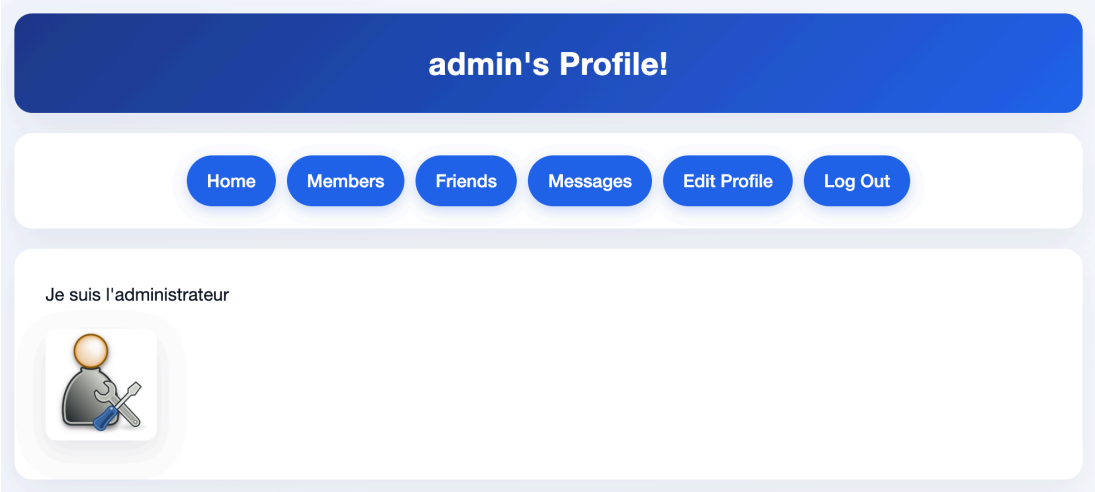
```php
function redirect($url, $statusCode = 303)
{
    header('Location: ' . $url, true, $statusCode);
    die();
}
```

Voici également quelques captures d'écran :

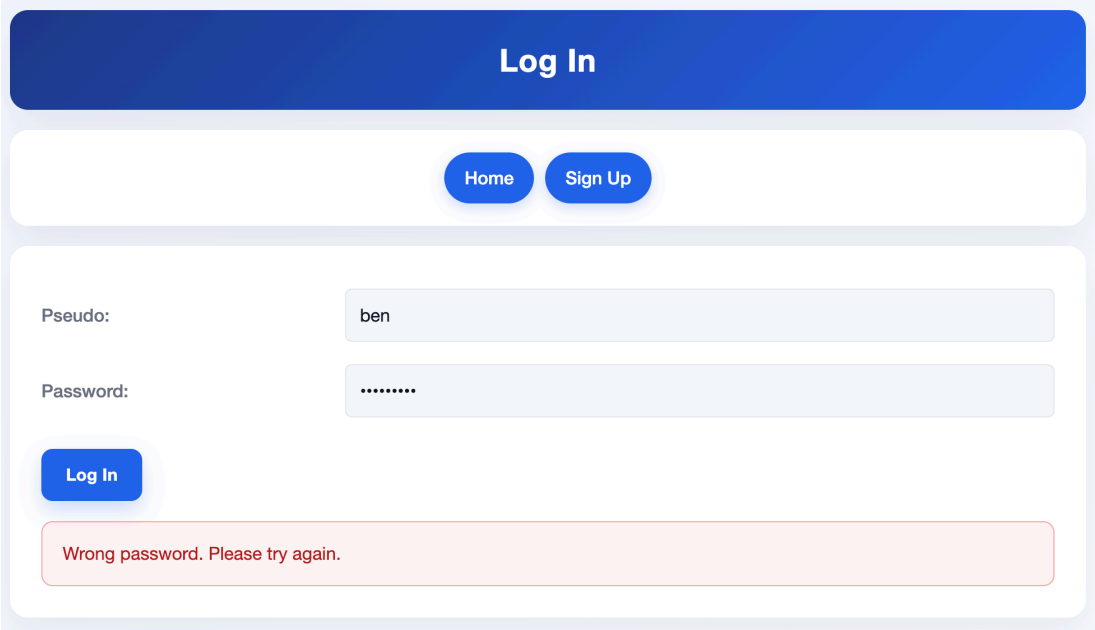
- Page de login



- Login valide (par exemple admin/admin) : on est dirigé vers le profil de l'utilisateur



- Erreur dans le mot de passe :



- Erreur dans le login :

